МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет інформаційних технологій
Кафедра комп’ютерної інженерії та інформаційних систем

Лабораторна робота №1
з дисципліни
«Кросплатформне програмування»

на тему:
«Базові операції у Java: змінні, методи та класи»

Виконав:
студент 4 курсу, групи КІ2-23-1 _____ Турунтаєв Р. М
(Підпис)

Перевірив: _____ Денисюк Д. О
(Підпис)

Хмельницький 2026

**Тема:** Базові операції у Java: змінні, методи та класи.

**Мета:** Налаштувати середовище розроблення Java-проєктів і реалізувати консольну програму, у якій обчислення зосереджено в предметному класі. Результат має відтворюватися засобами Maven, проходити модульні тести та зберігатися у загальнодоступному репозиторії Git.

**Вимірювані результати навчання:**

У результаті виконання лабораторної роботи необхідно:

1. Ознайомитися з призначенням JDK, компілятора javac, байткоду та JVM.
2. Навчитися створювати та налаштовувати Java-проєкти з використанням Apache NetBeans і Maven.
3. Реалізувати програму з використанням змінних, методів, класів та конструктора.
4. Навчитися перевіряти коректність вхідних даних.
5. Виконати модульне тестування програми за допомогою JUnit.
6. Перевірити складання проєкту засобами Maven та опублікувати його у Git-репозиторії.

**Індивідуальний варіант:** № 2 — Trip (розрахунок витрати палива та вартості поїздки).

## 1. Теоретичні відомості

Java — об'єктно-орієнтована мова програмування, яка дозволяє створювати програми для різних операційних систем. Вихідний код програми компілюється у байткод, який виконується віртуальною машиною Java (JVM).
Для розроблення програм використовується Java Development Kit (JDK), до складу якого входять компілятор javac та необхідні засоби розроблення.

Apache NetBeans — інтегроване середовище розроблення (IDE), яке дозволяє створювати, редагувати, компілювати та запускати Java-проєкти.

Apache Maven — інструмент автоматизації складання проєктів, який використовує файл pom.xml для визначення параметрів компіляції, залежностей і тестування.

JUnit — бібліотека для модульного тестування Java-програм. Вона дозволяє перевіряти правильність роботи окремих методів та виявляти помилки.

Git — система контролю версій, яка використовується для збереження історії змін проєкту. GitHub забезпечує розміщення Git-репозиторіїв і доступ до їхнього вмісту.

### 1.1. Вхідні дані та обмеження індивідуального завдання

У програмі використовуються три вхідні параметри типу double: відстань поїздки, кількість витраченого палива та ціна одного літра палива. Усі параметри повинні бути більшими за нуль та мати скінченні числові значення. Введення нульових, від'ємних значень, NaN або нескінченності є недопустимим. У разі порушення зазначених умов конструктор класу Trip генерує виняток IllegalArgumentException.

Данні, які використовувались в ході виконання лабораторної, зазначені у таблиці 1.

Таблиця 1.1 - Вхідні дані програми Trip

| Параметр | Тип | Одиниця вимірювання | Допустимі значення |
|---|---|---|---|
| distanceKm | double | км | > 0 |
| fuelLitres | double | л | > 0 |
| pricePerLitre | double | грош. од./л | > 0 |

### 1.2. Математичні формули розрахунку

Для обчислення показників поїздки використовувались такі формули:

1. Витрата палива на 100 км:

$$Q_{100} = \frac{Q}{S} * 100$$

де: $Q_{100}$ – витрата палива на 100 км, л/100 км; Q – кількість витраченого палива; S – відстань поїздки, км.

2. Загальна вартість палива:

$$C = Q * P$$

де: C – загальна вартість палива, грош. од.; P – ціна одного літра палива, грош. од./л.

3. Вартість палива на 100 км:

$$C_{100} = \frac{C}{S} * 100$$

де: $C_{100}$ – вартість палива на 100 км, грош. од./100 км.

Спочатку визначається витрата палива на 100 км шляхом ділення кількості витраченого палива на пройдену відстань із подальшим множенням на 100. Далі обчислюється загальна вартість палива як добуток його кількості та ціни одного літра. На завершення визначається вартість палива на 100 км шляхом ділення загальної вартості на відстань поїздки з подальшим множенням на 100.

Результати обчислень виводяться з точністю до двох знаків після десяткової крапки.

## 2. Хід роботи

### 2.1. Налаштування середовища розробки

Для виконання лабораторної роботи було встановлено Eclipse Temurin JDK 25 LTS та інтегроване середовище розроблення Apache NetBeans IDE 31.

Після встановлення JDK було налаштовано системні змінні середовища JAVA_HOME та PATH. Для перевірки правильності встановлення Java у командному рядку Windows виконано команди:

```
java --version
javac --version
```

У результаті перевірки було підтверджено використання JDK версії 25.0.4.1.

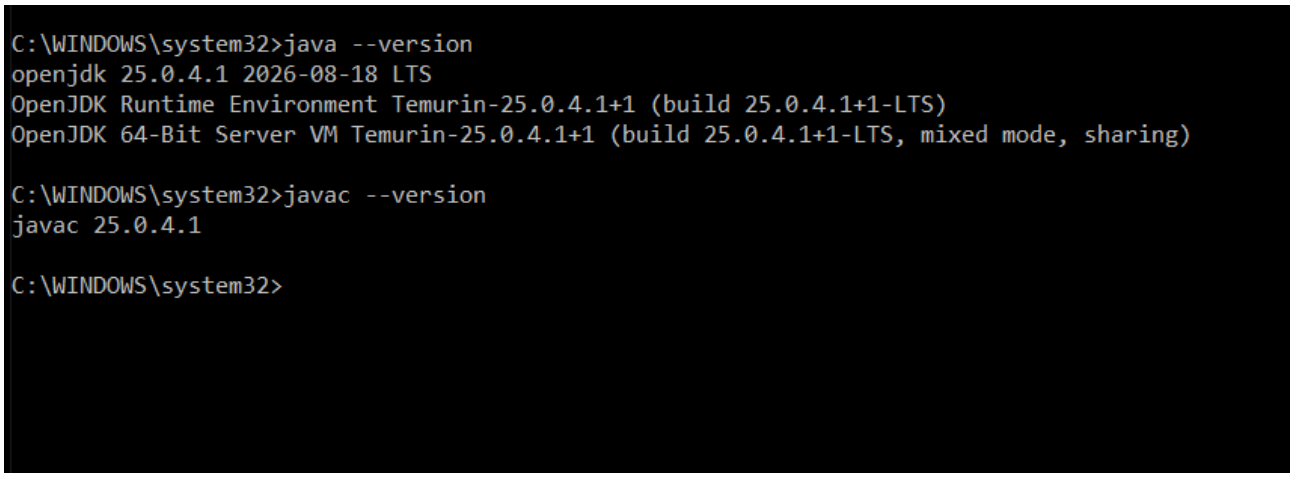



Рисунок 2.1. – Перевірка версії JDK у командному рядку Windows

Далі було запущено Apache NetBeans IDE 31 та перевірено його конфігурацію. Для складання проєкту використовується Apache Maven 3.9.16, а цільову версію компіляції встановлено на Java 21.

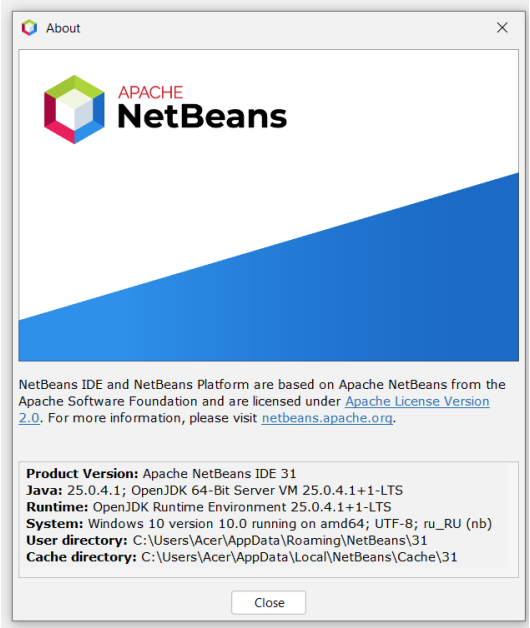



Рисунок 2.2. - Відомості про встановлене середовище Apache NetBeans IDE 31

## 2.2. Створення та налаштування Maven–проєкту

Після налаштування середовища розроблення в Apache NetBeans IDE 31 було створено новий Java-проєкт із використанням системи складання Apache Maven. Проєкт отримав назву Lab-01-Trip, а для розміщення вихідного коду було створено пакет `ua.khnu.kpp.lab1`.

Для налаштування проєкту було відредаговано файл pom.xml, у якому визначено цільову версію Java 21, бібліотеку JUnit Jupiter 6.1.3 та плагіни Maven Compiler Plugin і Maven Surefire Plugin.

Після збереження конфігурації було створено класи `Main.java` і `Trip.java`, а також тестовий клас `TripTest.java`.




Рисунок 2.3. - Структура Maven-проєкту Lab-01-Trip

Основні файли проєкту мають таке призначення:

- pom.xml — містить налаштування Maven, залежності та параметри складання проєкту.
- Main.java — відповідає за зчитування вхідних даних та виведення результатів.
- Trip.java — реалізує перевірку вхідних параметрів і математичні обчислення.
- TripTest.java — містить модульні тести для перевірки правильності обчислень.
- mvnw і mvnw.cmd — забезпечують запуск Maven Wrapper.
- submission.properties — містить номер лабораторної роботи та індивідуального варіанта.
- .gitignore — визначає файли й каталоги, які не потрібно додавати до Git-репозиторію.
- README.md — містить опис проєкту та інструкції щодо його складання, тестування й запуску.

### 2.3. Реалізація предметного класу Trip

Для виконання індивідуального завдання №2 було створено клас Trip.java, який призначений для обчислення витрати палива та вартості поїздки.

У класі оголошено три поля типу double: distanceKm, fuelLitres та pricePerLitre. Для їх ініціалізації реалізовано конструктор, який перевіряє коректність переданих значень. Якщо хоча б один параметр є недопустимим, генерується виняток IllegalArgumentException. Програмний код класу Trip.java наведено в лістингу 2.1.

Лістинг 2.1 – Програмний код класу Trip.java

```
package ua.khnu.kpp.lab1;

public final class Trip {

    private final double distanceKm;
    private final double fuelLitres;
    private final double pricePerLitre;

    public Trip(double distanceKm, double fuelLitres,
                double pricePerLitre) {

        if (!Double.isFinite(distanceKm) || distanceKm <= 0
                || !Double.isFinite(fuelLitres) || fuelLitres <= 0
                || !Double.isFinite(pricePerLitre) || pricePerLitre <= 0) {
            throw new IllegalArgumentException(
                    "All values must be finite and positive");
        }

        this.distanceKm = distanceKm;
        this.fuelLitres = fuelLitres;
        this.pricePerLitre = pricePerLitre;
    }

    public double fuelConsumptionPer100Km() {
        return (fuelLitres / distanceKm) * 100;
    }

    public double totalFuelCost() {
        return fuelLitres * pricePerLitre;
    }

    public double costPer100Km() {
        return (totalFuelCost() / distanceKm) * 100;
    }
}
```

Усі обчислення виконуються безпосередньо в методах класу Trip, без використання консольного введення або виведення. Методи повертають результати типу double без попереднього округлення.

Таким чином, клас Trip відповідає за збереження вхідних параметрів, перевірку їх допустимості та виконання необхідних математичних обчислень.

### 2.4. Реалізація головного класу Main

Для забезпечення взаємодії користувача з програмою було створено головний клас Main.java, який містить метод main().

Введення даних реалізовано за допомогою класу Scanner. Програма послідовно зчитує три числові значення: відстань поїздки, кількість витраченого палива та ціну одного літра палива. Після перевірки введених даних створюється об'єкт класу Trip, за допомогою якого виконуються необхідні обчислення.

Результати виводяться у форматі RESULT_N=%.2f із двома знаками після десяткової крапки. У разі некоректного введення програма виводить повідомлення ERROR. Програмний код головного класу наведено в лістингу 2.2.

Лістинг 2.2 – Програмний код класу Main.java

```
package ua.khnu.kpp.lab1;

import java.util.Locale;
import java.util.Scanner;

public final class Main {

    private static final String ERROR = "ERROR";

    private Main() {
    }

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in)
                .useLocale(Locale.ROOT);

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
```

```
        double distanceKm = input.nextDouble();

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
        double fuelLitres = input.nextDouble();

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
        double pricePerLitre = input.nextDouble();

        if (input.hasNext()) {
            System.out.println(ERROR);
            return;
        }

        try {
            Trip trip = new Trip(distanceKm, fuelLitres, pricePerLitre);

            System.out.printf(Locale.ROOT,
                    "RESULT_1=%.2f%n",
                    trip.fuelConsumptionPer100Km());

            System.out.printf(Locale.ROOT,
                    "RESULT_2=%.2f%n",
                    trip.totalFuelCost());

            System.out.printf(Locale.ROOT,
                    "RESULT_3=%.2f%n",
                    trip.costPer100Km());

        } catch (IllegalArgumentException exception) {
            System.out.println(ERROR);
        }
    }
}
```

Отже, клас Main відповідає лише за обробку введення та виведення даних, тоді як усі математичні обчислення виконуються в окремому класі Trip. Такий підхід дозволяє розділити функціональні обов'язки класів та спростити подальше тестування програми.

## 2.5. Модульне тестування програми

Для перевірки правильності роботи класу Trip було створено тестовий клас TripTest.java з використанням бібліотеки JUnit Jupiter 6.1.3.

У тестовому класі реалізовано три модульні тести:

- calculatesControlValues() — перевіряє правильність обчислень для контрольних значень, наведених у методичних матеріалах.
- calculatesAlternativeValues() — перевіряє результати обчислень для іншого допустимого набору вхідних даних.
- rejectsInvalidValues() — перевіряє виникнення винятку IllegalArgumentException у разі передавання недопустимих значень.

Для порівняння результатів типу double використовується метод assertEquals() із допустимою похибкою (10^{-9}). Для перевірки обробки некоректних параметрів застосовується метод assertThrows(). Програмний код тестового класу наведено в лістингу 2.3.

Лістинг 2.3 – Програмний код класу TripTest.java

```
package ua.khnu.kpp.lab1;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;

class TripTest {

    private static final double EPSILON = 1e-9;

    @Test
    void calculatesControlValues() {
        Trip trip = new Trip(420, 31.5, 59.9);

        assertEquals(7.50,
                trip.fuelConsumptionPer100Km(), EPSILON);
        assertEquals(1886.85,
                trip.totalFuelCost(), EPSILON);
        assertEquals(449.25,
                trip.costPer100Km(), EPSILON);
    }

    @Test
```

```
    void calculatesAlternativeValues() {
        Trip trip = new Trip(200, 12, 50);

        assertEquals(6.0,
                trip.fuelConsumptionPer100Km(), EPSILON);
        assertEquals(600.0,
                trip.totalFuelCost(), EPSILON);
        assertEquals(300.0,
                trip.costPer100Km(), EPSILON);
    }

    @Test
    void rejectsInvalidValues() {
        assertThrows(IllegalArgumentException.class,
                () -> new Trip(0, 10, 50));

        assertThrows(IllegalArgumentException.class,
                () -> new Trip(200, -5, 50));

        assertThrows(IllegalArgumentException.class,
                () -> new Trip(200, 10, Double.NaN));
    }
}
```

### 2.6. Перевірка складання та тестування проєкту

Після реалізації програмного коду та модульних тестів було виконано перевірку складання Maven-проєкту за допомогою Maven Wrapper.

Для розроблення, складання та тестування програми використовувалися такі програмні засоби:

Таблиця 2.1. - Використане програмне забезпечення

| Програмний засіб | Версія |
|---|---|
| Eclipse Temurin JDK | 25.0.4.1 LTS |
| Apache NetBeans IDE | 31 |
| Apache Maven | 3.9.16 |
| Java (цільова версія) | 21 |
| JUnit Jupiter | 6.1.3 |
| Maven Compiler Plugin | 3.15.0 |
| Maven Surefire Plugin | 3.5.5 |

Для перевірки проєкту в командному рядку Windows було виконано команду:

```
mvnw.cmd -B -ntp clean verify
```

Команда clean видаляє результати попереднього складання, а verify запускає необхідні етапи компіляції, тестування та перевірки проєкту.

Під час виконання команди було успішно скомпільовано вихідний код програми, запущено модульні тести JUnit та створено JAR-файл проєкту.

За результатами перевірки всі три модульні тести завершилися успішно:

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

Процес складання завершився повідомленням BUILD SUCCESS, що підтверджує успішне складання проєкту та проходження реалізованих тестів.

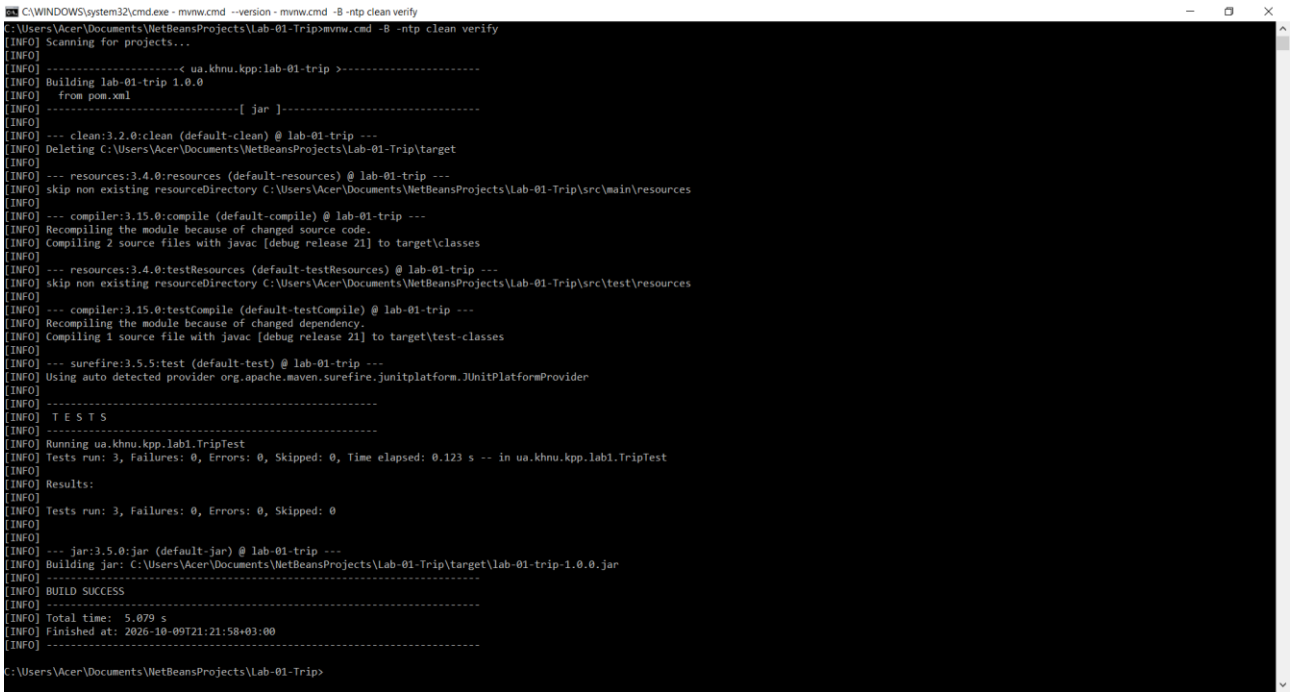



Рисунок 2.4 – Результат складання та тестування Maven-проєкту

## 2.7. Результати тестування програми

Для перевірки правильності роботи програми було використано три тестові сценарії: контрольний, альтернативний допустимий та негативний. Результати перевірки наведено в таблиці 2.3.

Таблиця 2.2. - Результати тестування програми Trip

| Тест | Вихідні дані | Очікуваний результат | Фактичний результат |
|---|---|---|---|
| Контрольний | 420; 31.5; 59.9 | 7.50; 1886.85; 449.25 | 7.50; 1886.85; 449.25 |
| Альтернативний | 200; 12; 50 | 6.00; 600.00; 300.00 | Тест пройдено; значення відповідають очікуваним |
| Негативний | 0; 10; 50 | IllegalArgumentException | Виняток підтверджено тестом JUnit |
| Негативний | 200; -5; 50 | IllegalArgumentException | Виняток підтверджено тестом JUnit |
| Негативний | 200; 10; NaN | IllegalArgumentException | Виняток підтверджено тестом JUnit |

За результатами виконання модульних тестів усі перевірки завершилися успішно. Фактичні результати контрольного набору збіглися з очікуваними, а альтернативний набір підтвердив правильність обчислень для інших допустимих значень. Негативні перевірки підтвердили, що конструктор класу Trip відхиляє недопустимі параметри шляхом генерації винятку IllegalArgumentException.

## 2.8. Розміщення проєкту в Git-репозиторії

Для збереження історії змін та забезпечення доступу до результатів лабораторної роботи було використано систему контролю версій Git. У каталозі проєкту було ініціалізовано локальний Git-репозиторій та створено гілку main. Після додавання вихідного коду, конфігураційних файлів і модульних тестів виконано початковий коміт.

Далі на платформі GitHub було створено власний публічний репозиторій, до якого завантажено файли Maven-проєкту:

https://github.com/neubaguette/lab-01-trip.git

Основна гілка: *main*. Тег завершеної лабораторної роботи: lab-01. Ідентифікатор завершального коміту (SHA): зазначається під час подання роботи.

**Висновок**

Під час виконання лабораторної роботи №1 було налаштовано середовище розроблення Java з використанням Eclipse Temurin JDK 25 та Apache NetBeans IDE 31.

Відповідно до індивідуального варіанта №2 реалізовано консольну програму Trip, яка дозволяє обчислювати витрату палива на 100 км, загальну вартість витраченого палива та його вартість на 100 км.

Для перевірки правильності обчислень було створено три модульні тести JUnit. Усі тести завершилися успішно, а складання проєкту засобами Maven виконано без помилок, що підтверджено результатом BUILD SUCCESS.

Також було використано систему контролю версій Git та опубліковано проєкт у власному репозиторії GitHub.

У результаті виконання роботи було закріплено практичні навички створення Java-класів, використання методів і конструкторів, перевірки вхідних даних, модульного тестування та керування версіями програмного проєкту.